CortexAI Authentication System Documentation

This document explains the setup, configuration, variables, functions, and the detailed file-to-file execution workflow showing how authentication is handled, validated, and propagated across CortexAI.

---

1. Variables & Functions Dictionary

Frontend

- `googlelogin` (function in `home.jsx`):
  - **What it does**: Triggers Google Sign-in.
  - **How it does it**: Launches Firebase's popup dialog via `signInWithPopup(auth, googleProvider)`, retrieves the authenticated credentials, and requests the short-lived Google ID Token string via `getIdToken()`.
- `handlelogin` (function in `home.jsx`):
  - **What it does**: Submits the Google ID Token to the Gateway.
  - **How it does it**: Posts the token payload to `/auth/login`. On success, it receives the logged-in user profile and dispatches `setUserdata` to save it globally.
- `getuser` (function in `App.jsx`):
  - **What it does**: Re-authenticates the user on page loads.
  - **How it does it**: Calls the feature helper `getcurrentuser()` which makes a `GET` request to `/me`. On success, dispatches profile data to Redux.
- `logout` (function in `logout.js`):
  - **What it does**: Logs out the active user session.
  - **How it does it**: Performs a request to the `/auth/logout` endpoint to clear the server session.
- `userData` (state variable in `userslice.js`):
  - **What it stores**: Holds the active user's profile details (`_id`, `name`, `email`, `avatar`) inside the Redux store.

Gateway

- `protect` (middleware in `auth.middleware.js`):
  - **What it does**: Restricts access to secured routes.
  - **How it does it**: Reads `sessionId` from the browser's incoming `session` cookie, verifies it against the Redis cache (`session-${sessionId}`), and sets the parsed user profile object on `req.user`.
- `getCurrentuser` (controller in `user.controller.js`):
  - **What it does**: Returns the active session profile data.
  - **How it works**: Reads `req.user` context populated by `protect` and responds with it.
- `proxyWithHeader` (utility in `proxywithheader.js`):
  - **What it does**: Decorates downstream microservice proxies.
  - **How it works**: Takes the user's database ID (`req.user.userid`) and injects it as an `x-user-id` HTTP header.

Auth Microservice

- `login` (controller in `auth.controller.js`):
  - **Variables**:
    - `token`: Incoming Google ID Token payload.
    - `decoded`: Verified profile data from Firebase Admin SDK (`verifyIdToken`).
    - `user`: MongoDB user record found or registered.
    - `sessionid`: Cryptographically secure random UUID (`crypto.randomUUID()`).
  - **How it works**: Verifies JWT, updates MongoDB, caches session string in Redis for 7 days under key `session-${sessionid}`, and sets a secure HttpOnly cookie `session`.
- `logout` (controller in `auth.controller.js`):
  - **Variables**:
    - `sessionid`: Cookie identifier retrieved from request cookies.
  - **How it works**: Wipes the `session-${sessionid}` key from Redis and clears the cookie.

---

2. Global Integration (Where, When, & How Auth Data/Functions are Used)

The authentication system is integrated throughout the application to secure endpoints and sync user state:

A. Login & Session Creation (`googlelogin` & `handlelogin`)

- **Where**: Triggered inside the Google Button click handler in `home.jsx`.
- **When**: Runs when an unauthenticated user accesses the page and signs in.
- **How**: Fetches the Google JWT, posts it to the backend via Axios, and dispatches the returned profile to the Redux store to log in the user.

B. Route Protection (`protect`)

- **Where**: Imported and run inside `gateway/index.js`.
- **When**: Registered as middleware on endpoints `/me`, `/chat`, and `/agent`.
- **How**: Halts incoming requests to check cookies and Redis cache. If validation fails, it returns `401 Unauthorized` before reaching any backend logic.

C. Downstream Identity Verification (`proxyWithHeader`)

- **Where**: Used in `gateway/index.js` route proxy definitions.
- **When**: Applied when forwarding client requests from the Gateway to microservices.
- **How**: Extracts `req.user.userid` and attaches it as `x-user-id` in the proxy headers.
- **Downstream Usage**:
  - **Chat Service** (`chat.controller.js`): Extracts the header to query and update chat conversations for that specific user.
  - **Agent Service** (`agent.controller.js`): Extracts the header to identify the user before starting AI flows.

D. Client Profile Sync (`userData` & `setUserdata`)

- **Where**: Imported in `App.jsx`, `home.jsx`, and `sidebar.jsx`.
- **When**: Evaluated and modified inside component layouts.
- **How**:

- **`home.jsx`**: Checks `userData` via `useSelector`. If populated, it closes the login popup.
- **`sidebar.jsx`**: Reads `userData` to render the user's name, email, and Google avatar image at the bottom of the drawer.
- **Logout handler**: On logout click, it calls `logout()` and dispatches `setUserdata(null)` to trigger the login popup again.

---

3. File-to-File Execution Workflow

Below is the step-by-step path the code execution takes from file to file during login:

```
[Trigger] home.jsx (Frontend)
   │  ──> User clicks Google login, triggering googlelogin() popup.
   │  ──> Extracts Google JWT token into "token".
   │  ──> Posts token payload to Gateway URL at "/auth/login".
   ▼
[Route Proxy] gateway/index.js
   │  ──> Matches "/auth" prefix and proxies request to Auth Service (Port 8001).
   ▼
[Router] auth.routes.js (Auth Service)
   │  ──> Forwards post request to login controller inside auth.controller.js.
   ▼
[Verification] auth.controller.js
   │  ──> Verifies JWT via Firebase Admin getAuth().verifyIdToken(token).
   │  ──> Queries MongoDB user.model.js to find/register User profile.
   │  ──> Generates a secure session ID UUID: "sessionid".
   │  ──> Caches session JSON in Redis: redis.set("session-" + sessionid, userJSON, "EX",
   │  ──> Attaches secure cookie "session" containing the UUID to HTTP response headers.
   │  ──> Returns 200 OK with user profile payload to client.
   ▼
[State Sync] home.jsx
   │  ──> Receives user profile payload.
   │  ──> Dispatches: dispatch(setUserdata(data)) to redux/userslice.js.
   ▼
[UI Updates] home.jsx & sidebar.jsx
      ──> Detects userData is populated, closes login modal, and renders workspace.
```